Optimising attention on Apple M1

Code: https://github.com/lazarsalt1441/CS6886_Assignment5

At $N = 4096, d = 64$, naive attention takes 1734.581 ms. The best single-threaded variant takes 58.175 ms (29.82× faster), and 8 OpenMP threads take 13.578 ms (127.75× faster).

Q1. Machine and peak calculations

The machine is a MacBook Air (MacBookAir10,1), with an Apple M1 and 16 GiB unified memory. Native `sysctl` confirms four performance (P) cores and four efficiency (E) cores, eight physical/logical cores and no SMT. All timings are native arm64. The exact macOS and Homebrew LLVM versions are recorded in `machine.json`; compilation uses `-O3 -mcpu=apple-m1 -fopenmp-simd`; only the OpenMP variant enables parallel OpenMP regions with `-fopenmp`.

Property	Value / evidence
Core types	Firestorm P + Icestorm E; 4 of each (native probe; Apple [1])
SIMD / registers	128-bit NEON, 4 FP32 lanes; 32 architectural vector registers
FMLA throughput	P: 4/cycle; E: 2/cycle (instruction microbenchmarks [2])
Clock reference	P: 3.204 GHz; E: 2.064 GHz, platform operating points [3]
Base / sustained clock	DVFS; collected sysctl does not expose run-time core clocks
L1D / L1I per P core	128 KiB / 192 KiB (native perflevel0 sysctl)
L1D / L1I per E core	64 KiB / 128 KiB (native perflevel1 sysctl)
L2 clusters	12 MiB shared by 4 P cores; 4 MiB shared by 4 E cores
L3 / cache line	Conventional L3 not reported; system cache unprobed / 128 B
Memory	16 GiB unified; LPDDR4X-4266 platform reference [4]

An FMA counts as two operations. Using the platform maximum operating points, the reference peaks are

$$P_P = 3.204 \times 4 \times 4 \times 2 = 102.528 \text{ GFLOP/s},$$
$$P_E = 2.064 \times 2 \times 4 \times 2 = 33.024 \text{ GFLOP/s},$$
$$P_{all} = 4P_P + 4P_E = 542.208 \text{ GFLOP/s}.$$

These are conditional architectural upper bounds, *not measured sustained clocks*. The P operating point appears as a turbo entry in the Linux/Asahi platform description; its Linux availability does not establish which macOS frequency was used. A fixed base/all-core frequency cannot be reported from this run. Percent-of-peak uses P_P for serial variants; OpenMP uses $P(t) = \min(t, 4)P_P + \max(0, t-4)P_E$, a P-first reference model, not verified core placement.

For the platform memory model, use a 128-bit aggregate interface (two 64-bit-equivalent widths): $B_{ref} = 4266 \times 16/1000 = 68.256$ GB/s. Memory type/rate come from platform identification [4]; the bus width is an explicit model assumption, not a probe of physical LPDDR subchannels. A 384 MiB triad measures useful bandwidth as $B = 12n/t$ (two reads plus one FP32 write), excluding cached-store write allocation. The one-thread rate is 58.170 GB/s; the best is 58.170 GB/s at 1 threads; eight threads give 47.935 GB/s. Using measured bandwidth, the reference ridge points are $P_P/B_1 = 1.763$ and $P_{all}/B_8 = 11.311$ FLOP/B. With theoretical B_{ref}, they are 1.502 and 7.944 FLOP/B.

Q2. Naive baseline

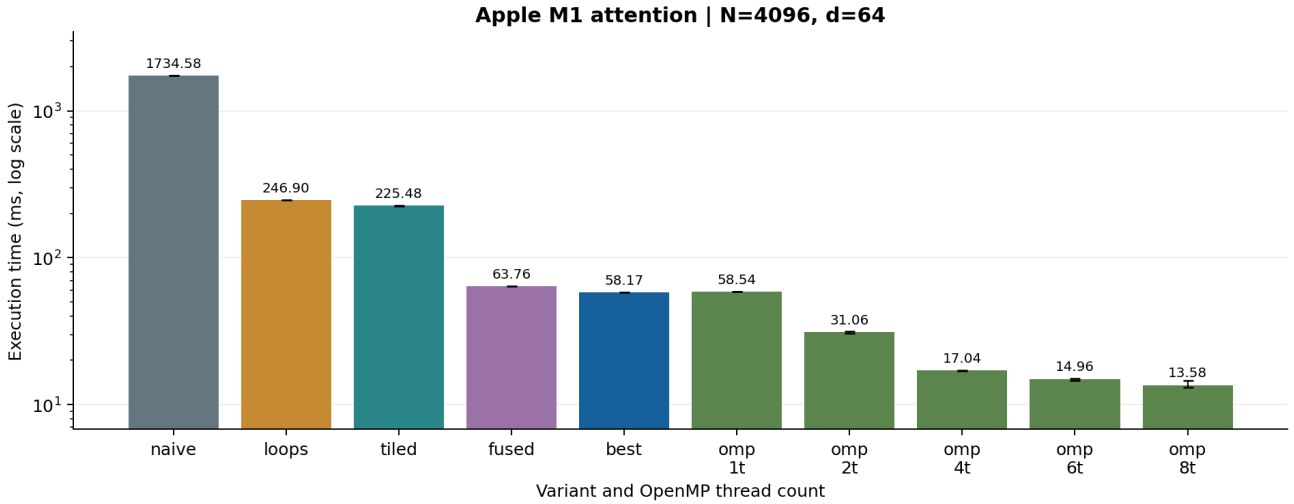
Variant	N	d	Time (ms)	GFLOP/s	% peak*	vs naive
naive	1024	64	105.700	2.54	2.48	1.00×
naive	2048	64	427.157	2.51	2.45	1.00×
naive	4096	64	1734.581	2.48	2.42	1.00×
naive	4096	128	3715.327	2.31	2.26	1.00×

The common work count is $F = 4N^2d$: $2N^2d$ for QK^T and $2N^2d$ for the value product. Reported throughput is $F/(10^9 t)$. Softmax, division, key transposition and allocation remain in the time but not the useful-work numerator, following the starter driver.

At $d = 64$, doubling N multiplies time by 4.04 and 4.06, close to the quadratic prediction of 4. At $N = 4096$, doubling d multiplies time by 2.14: the products double in work, while softmax still handles N^2 scores. Different cache/stride behaviour also matters.

The baseline computes each score as a separate dot reduction. Strict FP32 summation leaves a dependent addition chain. Its value product uses i, k, j loops, so $V[jd + k]$ is read with a 256 B stride at $d = 64$ and 512 B at $d = 128$. The same probability row is traversed for every output feature. This wastes locality and exposes reductions rather than independent contiguous SIMD updates. These are code-based explanations of naive; its phase shares and hardware cache-miss counters were not measured. The optimised kernels have separate phase profiles.

Protocol. Every original starter driver is preserved. Each invocation performs one warm-up and takes the median of five timed calls; the CSV is the median of three independently warmed invocations per configuration. Only `attention()` is timed; file I/O is excluded, while all temporary allocations and packing are included. Configurations run sequentially. macOS owns P/E placement; no CPU affinity or fixed core frequency is claimed. Three-trial ranges on the time plots are variability indicators, not confidence intervals.



The full CSV contains every input/variant/thread configuration. All four required per-input time plots are supplied. One noisy invocation cannot change a row to its best-case time: the report consistently uses the three-invocation median.

* Peak percentages refer to the Q1 reference architecture/frequency model. GB and GFLOP use decimal units; KiB/MiB/GiB use binary units.

Q3. Single-threaded variants

Variant	N	d	Time (ms)	GFLOP/s	% peak*	vs naive
loops	1024	64	15.514	17.30	16.88	6.81×
loops	2048	64	60.892	17.63	17.20	7.01×
loops	4096	64	246.901	17.40	16.97	7.03×
loops	4096	128	394.217	21.79	21.25	9.42×
tiled	1024	64	13.550	19.81	19.32	7.80×
tiled	2048	64	56.639	18.96	18.49	7.54×
tiled	4096	64	225.478	19.05	18.58	7.69×
tiled	4096	128	399.539	21.50	20.97	9.30×
fused	1024	64	4.553	58.96	57.50	23.22×
fused	2048	64	15.995	67.13	65.47	26.71×
fused	4096	64	63.762	67.36	65.70	27.20×
fused	4096	128	116.136	73.96	72.14	31.99×
best	1024	64	3.934	68.23	66.55	26.87×
best	2048	64	14.710	72.99	71.19	29.04×
best	4096	64	58.175	73.83	72.01	29.82×
best	4096	128	105.477	81.44	79.43	35.22×

(a) Reordered loops. Transpose keys to $T[k, j]$ and use i, k, j score loops. Adjacent key positions become independent SIMD lanes. For SV, use i, j, k so each probability updates contiguous output features from a contiguous row of V. This removes strided column access, but repeatedly updates score/output accumulators and retains the full score matrix.

(b) Cache/register tiling. Use 32×128 outer query/key tiles around 4×32 local accumulator tiles. Four query rows reuse each key/value vector; complete reductions are kept in accumulators before storing results. The measured table gives the actual shape-dependent benefit; local-array/control overhead can offset cache reuse.

(c) Blocked online fusion. Process 48 queries and 128 keys at a time. The score tile occupies 24 KiB instead of materialising the full $N \times N$ matrix. Keys are packed in key-block-major order, so the inner reduction stride is 128 rather than N. Four query rows reuse each vector of keys and values in register microkernels. For each row retain a running maximum m, denominator l and unnormalised output u. For a score tile s, set

$$m' = \max(m, \max s), \quad a = e^{m-m'}, \quad p = e^{s-m'}, \quad l' = al + \sum p, \quad u' = au + pV.$$

Initialise $l = u = 0$, then divide u by l once after the final key tile. Both the denominator and the old output numerator are rescaled when the maximum increases. Two independent four-lane NEON exponential evaluations handle eight scores per loop.

(d) Best M1 implementation. Use the same online kernel with configurable query, key and register tiles. The optional native tuner measures ten combinations of 24–96 query rows, 64–256 keys, and four or six register rows. It selects one global setting by geometric mean over all four supplied shapes using two reversed-order rounds. The exact selected flags and trial timings are retained in the native evidence. Without tuning, best uses the conservative 48×128 / four-register-row setting. Packing, allocations and online rescaling remain inside the timed call. Tail paths handle dimensions that do not divide the register or cache tile sizes.

The remaining limits include FMA/load/broadcast throughput, softmax, packing, cache traffic and OS scheduling. Optional separate instrumented builds record phase shares and matrix throughput; these profiling timings are not substituted for uninstrumented CSV measurements.

The submitted native run selected a 48×64 query/key cache tile and 4 register rows, using 12 KiB of score scratch per worker. These settings are shared by best and OpenMP.

Paired old/new comparison at $N = 4096, d = 64$, fused: 240.963 to 63.724 ms (3.78×).

Paired old/new comparison at $N = 4096, d = 64$, best: 73.812 to 58.285 ms (1.27×). These separate three-trial comparisons alternate old/new order; they are not timings from different machines or substituted into the main table.

For best at $N = 4096, d = 64$, the separate instrumented profile attributes 80.32% to the two matrix products and 18.86% to softmax/output normalisation. Score and value products achieve 95.00 and 87.76 GFLOP/s. These figures include instrumentation overhead.

Q4. OpenMP

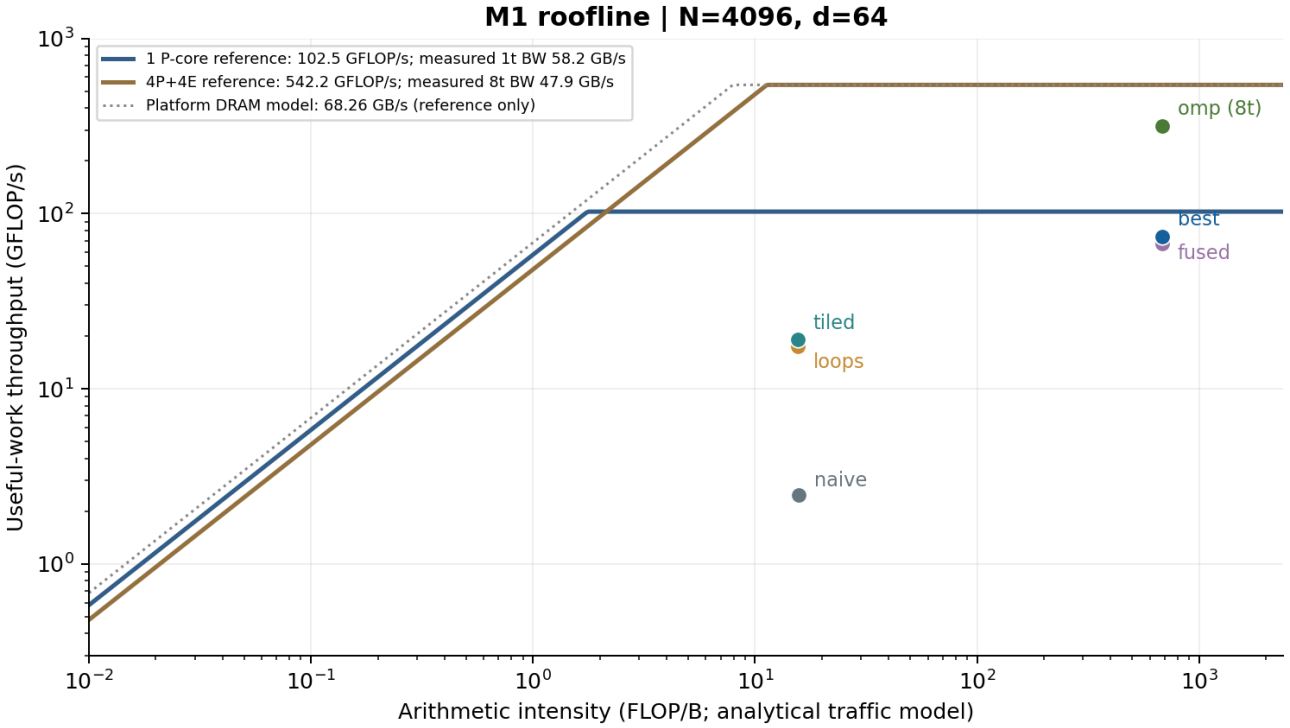
The same compute kernel and selected tile flags are used in `best` and `omp`. Packing and query processing share one parallel region, with a barrier after key packing. Query blocks use `schedule(dynamic,1)`: workers own private scratch and disjoint output rows, and read shared keys/values. Dynamic block assignment lets faster workers take more work on the asymmetric 4P+4E CPU. There is no cross-thread output reduction.

The controller disables dynamic team resizing and core binding. Thread placement remains OS-managed. Define $S_t = T_{omp,1}/T_{omp,t}$ and $E_t = S_t/t$, using the one-thread OpenMP implementation as the scaling baseline rather than the separately compiled serial binary.

N	d	Threads	Time (ms)	GFLOP/s	S_t	E_t (%)
1024	64	1	4.280	62.72	1.000	100.0
1024	64	2	1.997	134.42	2.143	107.2
1024	64	4	1.144	234.65	3.741	93.5
1024	64	6	1.130	237.55	3.788	63.1
1024	64	8	1.059	253.48	4.042	50.5
2048	64	1	14.741	72.84	1.000	100.0
2048	64	2	7.754	138.48	1.901	95.1
2048	64	4	4.105	261.57	3.591	89.8
2048	64	6	3.607	297.68	4.087	68.1
2048	64	8	3.393	316.46	4.345	54.3
4096	64	1	58.537	73.37	1.000	100.0
4096	64	2	31.056	138.30	1.885	94.2
4096	64	4	17.043	252.01	3.435	85.9
4096	64	6	14.956	287.17	3.914	65.2
4096	64	8	13.578	316.32	4.311	53.9
4096	128	1	105.389	81.51	1.000	100.0
4096	128	2	57.292	149.93	1.840	92.0
4096	128	4	31.719	270.81	3.323	83.1
4096	128	6	27.293	314.73	3.861	64.4
4096	128	8	24.851	345.66	4.241	53.0

For $N = 4096, d = 64$, eight threads yield 4.311× scaling and 53.9% efficiency, or 127.75× over naive. Eight identical full-speed cores would suggest an 8× line, but this CPU has four slower E cores, shared caches/memory and scheduling overhead. That line is context, not an achievable P-core-only target. The table records the actual per-shape ordering of thread counts, not which thread was on which core. Contention, dynamic scheduling, placement and thermal/DVFS effects are plausible contributors, not separately established causes.

Q5. Roofline



Analytical traffic model. Start with $16Nd$ bytes for reading Q, K, V and writing O. For materialised scores, add $16N^2$ bytes across stages: score write, softmax read/write, then value-product read. Softmax's internal row passes are assumed to hit cache. A transposed key buffer adds a conservative $8Nd$ bytes for its write/read. Fused row/block scratch is assumed cache-resident. Ignore write allocation and additional evictions. Hence these idealised intensities are not measured DRAM traffic and may overestimate actual intensity.

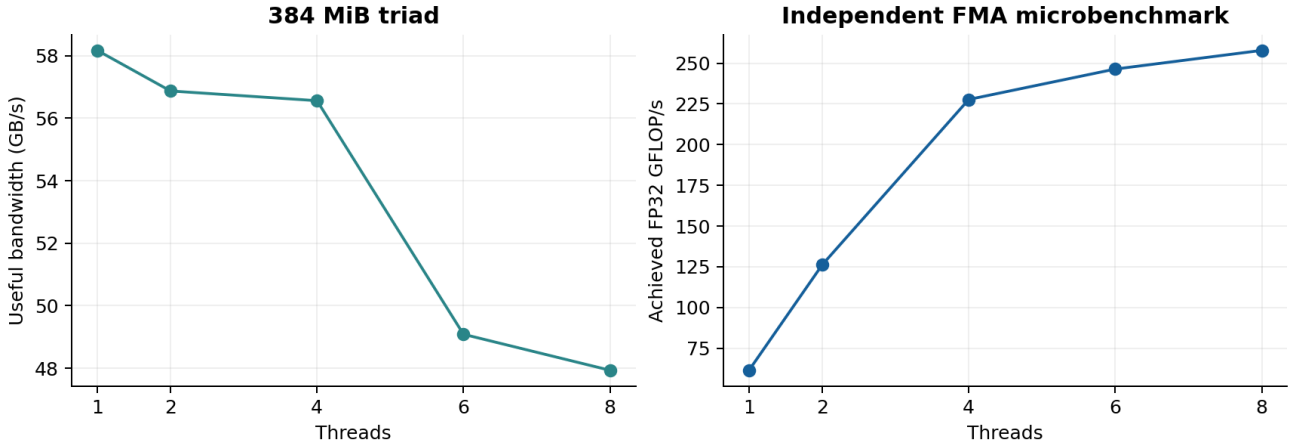
Variant	Modelled bytes	$I = F/B$ (FLOP/B)	GFLOP/s
naive	$16N^2 + 16Nd$	15.754	2.48
loops	$16N^2 + 24Nd$	15.634	17.40
tiled	$16N^2 + 24Nd$	15.634	19.05
fused	$24Nd$	682.667	67.36
best	$24Nd$	682.667	73.83
omp (8t)	$24Nd$	682.667	316.32

The rooflines use $\min(BI, P)$ with Q1's reference peak and either measured sustained bandwidth or the explicitly assumed platform memory model. Serial points belong against the one-P-core curve; the OpenMP point uses the 4P+4E curve. Compare each measured point with its corresponding ridge; a high modelled intensity alone does not establish a purely compute-bound kernel. Naive suffers reduction/stride/cache-level costs; loops exposes SIMD but repeats accumulator traffic; tiled reuses accumulators. Online fused/best retain bounded score tiles, reuse key/value vectors across queries, and vectorise softmax. OpenMP adds parallel query work.

Validation, auxiliary measurements and reproduction

Validation. All 36 non-reference input/configuration rows pass the required $\max|O - O_{naive}|/\max|O_{naive}| \leq 10^{-4}$, including all 20 OpenMP configurations. The worst supplied-input error is 6.24e-06. All 168 independent FP64 checks pass, with a worst error of 3.06e-05; these include tile-boundary tails, early/late maximum changes, constant/zero values, zero/uniform logits and large logits. SHA-256 matches all returned C/header sources and four original inputs to the packaged files. The original naive source and all starter drivers are preserved. `make check` rejects missing outputs, invalid headers/lengths, non-finite values and excessive error.

Vector softmax approximation. Subtract the row maximum, then reduce $x = n\ln 2 + r$ with $n = \mathrm{round}(x\log_2 e)$ and $|r| \leq \ln 2/2$. Evaluate $e^x \approx 2^n \sum_{k=0}^{7} r^k/k!$ with NEON FMAs. Values below -87 are set to zero (each contribution below 1.65×10^{-38}); scalar tails use `expf`. The first omitted polynomial term is below 5.2×10^{-9}, but final accuracy also includes FP32 range-reduction, FMA and summation rounding. Output checks establish tolerance on the tested cases, not exactness.



Threads	Triad GB/s	Achieved FMA GFLOP/s
1	58.170	61.473
2	56.871	126.232
4	56.561	227.705
6	49.086	246.372
8	47.935	257.838

The FMA test runs sixteen independent NEON chains, one warm-up and five measurements, counting $16 \times 4 \times 2$ FLOPs per iteration per thread. It reports achieved rates, not a hard roof: one attention shape exceeds its one-thread rate because instruction mix, timing and operating conditions differ. The triad's best bandwidth occurs at two threads; more workers add contention and scheduling costs rather than increasing the measured useful byte rate. No cache-miss, power or sustained-frequency counters were collected.

Reproduce. On the M1, install Homebrew LLVM/libomp/Python, then run `bash src/run_macos.sh`. It compiles the kernels, runs all configurations/checks and collects a result ZIP. `make all; make check` gives the starter-style workflow. `python3 src/finalize_mac.py` rebuilds this PDF, CSV/specification and plots from the included native evidence (requires Matplotlib, NumPy and pdflatex). Use `--data PATH` to supply another complete native run. Regeneration validates source hashes and raw medians.

References and run trace

[1] Apple, MacBook Air (M1, 2020) technical specifications.

https://support.apple.com/en-us/111883

[2a] Dougall Johnson, Apple Microarchitecture Research: Firestorm SIMD/FP, FMLA vector 4S (0.25 cycles per instruction).

https://dougallj.github.io/applecpu/firestorm-simd.html

[2b] Dougall Johnson, Icestorm SIMD/FP, FMLA vector 4S (0.5 cycles per instruction). These are primary reverse-engineering measurements, not Apple specifications.

https://dougallj.github.io/applecpu/icestorm-simd.html

[3] Asahi Linux contributors, Linux T8103 device tree: 3.204 GHz P turbo / 2.064 GHz E operating points; these are reference values, not observed macOS clocks.

https://github.com/torvalds/linux/blob/master/arch/arm64/boot/dts/apple/t8103.dtsi

[4] iFixit, M1 MacBook Pro and Air Teardowns: primary chip identification of LPDDR4X-4266. The report explicitly assumes a 128-bit aggregate interface for the theoretical bandwidth model.

https://www.ifixit.com/News/46884/m1-macbook-teardowns-something-old-something-new

[5] Apple, Tuning your code's performance for Apple silicon: asymmetric cores and scheduling.

https://developer.apple.com/documentation/apple-silicon/tuning-your-code-s-performance-for-apple-silicon/

[6] Assignment starter, commit 29f5b9bf4aa89f74f425b1797cc46cdb457f5569.

https://github.com/NMJ0/Assignment-5-Systems-for-Deep-learning